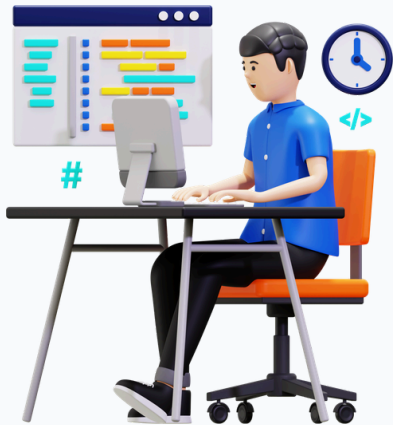


Authentication & JWT Tokens

Q.E.D. — Web Dev Bootcamp | Session 03



Why Do We Need Authentication?

Without auth, anyone can access anything. With auth, each user only accesses what belongs to them.

Concept

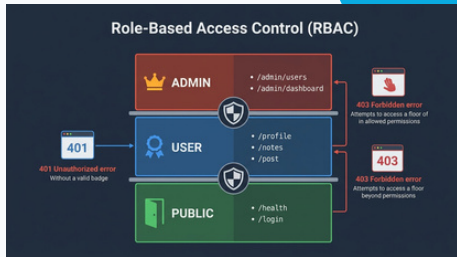
Meaning

Authentication "Who are you?" — Proving your identity.

Authorization "What can you do?" — Checking permissions.

Real-world analogy:

Authorization = Your badge lets you into Floor 3 but not the server room.



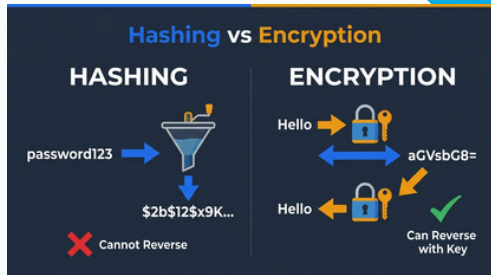
1. Hashing vs Encryption

Why? We store passwords as hashes so even if the database is stolen, no one can read them.

Feature	Hashing	Encryption
Direction	One-way	Two-way
Reversible?	No	Yes (with key)
Used for	Passwords	Messages, files
Algorithm	bcrypt	AES, RSA

Never use MD5 or SHA for passwords!

They are too fast — hackers can try billions per second. bcrypt is intentionally slow (~100ms/attempt).



1. Hashing in Python (passlib + bcrypt)

How? Use passlib library to hash and verify passwords.

During Signup (Hash)

```
from passlib.context import CryptContext

pwd_context= CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)

# the password
Hash = pwd_context.hash("password123")
# $2b$12$LJ3m5Kq...
```

During Login (Verify)

```
# Compare input with stored hash
pwd_context.verify(
    "password123", #userinput
    hashed         #fromdatabase
)
# True

pwd_context.verify(
    "wrong_pass",
    hashed
)
# False
```

You don't "decrypt" the hash. You hash the input again and compare the hashes.

2. JWT Tokens — Your Digital ID Card

Why? After login, the server gives you a token. You send it with every request to prove who you are.



2. Creating & Decoding JWT in Python

How? Use the python-jose library.

Create Token (after login)

```
from jose import jwt
from datetime import datetime, timedelta

SECR = "your-secret-key"
ET = "HS256"
ALGO = "HS256"

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(
        minutes=30)
    to_encode["exp"] = expire
    return jwt.encode(to_encode, SECR,
        algorithm=ALGO)

# Usage:
token = create_access_token({"sub": "42"})
```

Decode Token (verify request)

```
from jose import jwt, JWTError

def decode_token(token: str):
    try:
        payload = jwt.decode(
            token,
            SECR,
            algorithms=[ALGO]
        )
        return payload
    except JWTError:
        raise HTTPException(
            status_code=401,
            detail="Invalid token"
        )
```

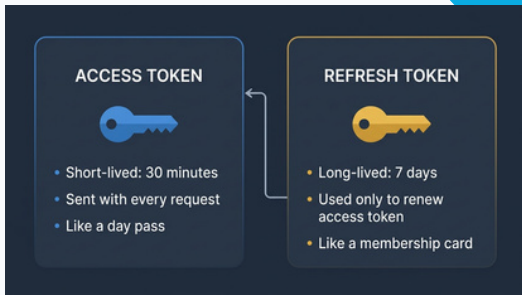
3. Access Token vs Refresh Token

Why? Short access tokens limit damage if stolen. Refresh tokens prevent users from logging in every 30 minutes.

	Access	Refresh
Lifetime	30 min	7 days
Sent to	Every endpoint	Only /refresh
If stolen	30 min damage	More dangerous
Purpose	Prove identity	Renew access

The Flow:

Login → get both tokens → use access for requests → when it expires → use refresh to get a new access token → no re-login needed!



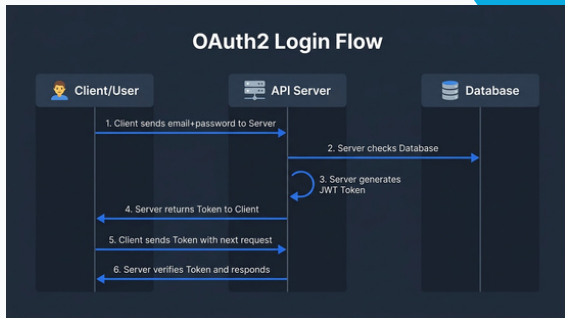
4. The OAuth2 Login Flow

How? This is the standard flow every modern API uses for authentication.

The 5 Steps:

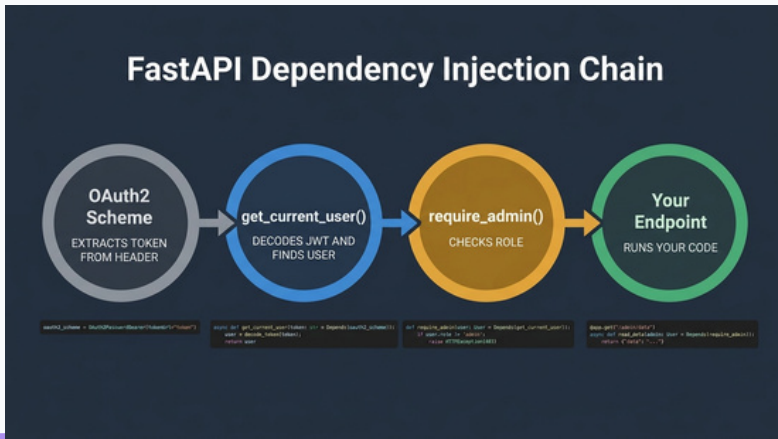
1. Client sends email + password to /auth/login
2. Server checks if user exists in the database
3. Server verifies the password hash (bcrypt)
4. Server generates JWT tokens (access + refresh)
5. Client stores the token and sends it with every future request via Authorization: Bearer <token> header

Security Tip: Always return the same error message for "wrong email" and "wrong password": "Invalid credentials". Don't help attackers guess which one is wrong!



5. FastAPI Dependencies — Depends()

Why? Run validation code **before** your endpoint. If it fails, the endpoint never runs.



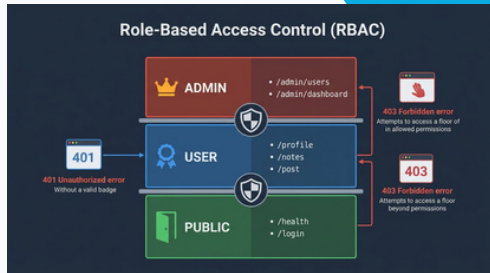
6. Role-Based Access Control (RBAC)

Why? Not all users are equal. Admins can delete users, regular users cannot.

Code	Meaning	Analogy
401	Unauthorized	"Show me your ID."
403	Forbidden	"I know you, but no."

Admin Dependency

```
def require_admin(  
    user = Depends(get_current_user)  
):  
    if user["role"] != "admin":  
        raise HTTPException(  
            status_code=403,  
            detail="Admin access required"  
        )
```



The Chain:

```
oauth2_scheme  
↓  
extracts token  
↓  
get_current_user  
↓  
decodes JWT  
↓  
require_admin  
↓  
checks role  
↓  
Your endpoint runs ✓
```

Auth Status Codes Cheat Sheet

Code	Title	When to use it	Who throws it?
200	OK	Login successful, profile returned.	You
201	Created	New user registered successfully.	You
401	Unauthorized	Wrong email/password, expired or missing token.	You
403	Forbidden	User is logged in but not admin.	You
409	Conflict	Email already registered (duplicate signup).	You
422	Unprocessable	Invalid input format (missing field, wrong type).	Pydantic

Pro Tip: For login failures, **always** return 401 with "Invalid credentials" — never tell the attacker

Quick Reference

The Building Blocks

- **bcrypt** — Hash passwords (one-way, slow, secure).
- **JWT** — Token = Header.Payload.Signature.
- **Access Token** — Short (30 min), sent with requests.
- **Refresh Token** — Long (7 days), renews access.

Libraries You Need

- `passlib[bcrypt]` — Password hashing
- `python-jose[cryptography]` —JWT
- `fastapi.security` — OAuth2PasswordBearer

FastAPI Auth Pattern

- **Public** — No `Depends()` anyone can call it.
- **Protected** — `Depends(get_current_user)`.
- **Admin Only** — `Depends(require_admin)`

Status Code Rules

- 201 for signup success
- 40 for “who are you?”
- 1 for “you can’t do this”
- 40 for duplicate email



Thank You